



FastAPI

Model Engineering

Prepared by WeCloudData

FastAPI Concepts

FastAPI 101

ML Application Example

Agenda.



What is it?

FastAPI Concepts



FastAPI is a modern, fast (high-performance), web framework for building APIs with Python 3.6+ based on standard Python type hints.

<https://fastapi.tiangolo.com/>

Facts:

- Released in 2018
- Intended for data science applications and web applications
- Used by big tech companies like Microsoft, Netflix and Uber

Advantages:

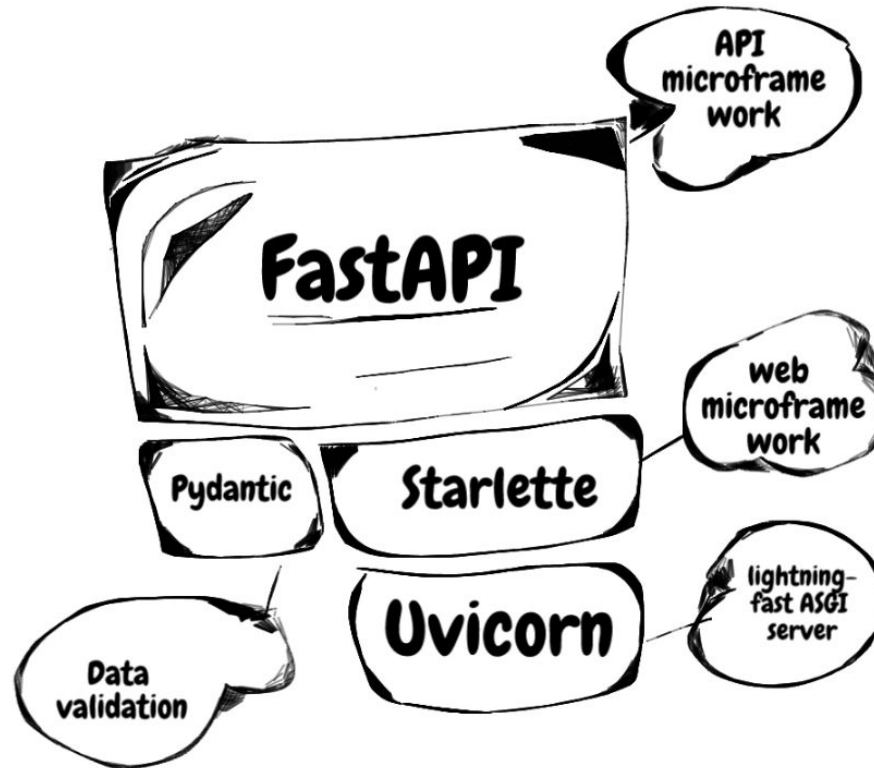
- High performance: speedy and robust
- Modern: Python 3.6+, type hints and annotations
- Open standards: OpenAPI, JSON Schema
- Automatic doc: SwaggerUI
- Developer experience: editor support



Architecture

FastAPI Concepts

- One of the fastest Python web frameworks available, only below Starlette and Uvicorn themselves (also used internally by FastAPI)



Hierarchy

FastAPI (uses Starlette)
an API microframework



Starlette (uses Uvicorn)
a web microframework



Uvicorn
an ASGI server



WSGI vs ASGI

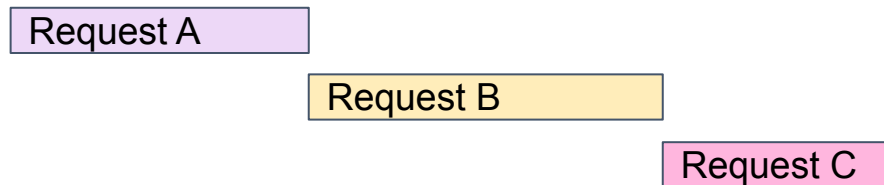
FastAPI Concepts

- WSGI and ASGI both specify the interface and sit in between the web server and a Python web application or framework.
- ASGI is relatively new but it has a clear performance advantage.
- **Flask** is WSGI-based and **FastAPI** is ASGI-based.

WSGI

(Web Server Gateway Interface)

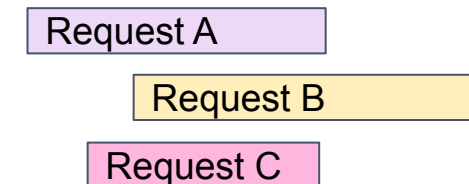
Synchronous request handling



ASGI

(Async Server Gateway Interface)

Asynchronous request handling





Starlette

FastAPI Concepts

- Starlette is a lightweight ASGI framework/toolkit, which is ideal for building high performance asyncio services.
- FastAPI is built on top of Starlette but with additional features for building APIs, with data validation, etc.



<https://www.starlette.io/>

- A lightweight HTTP web framework.
- WebSocket support.
- In-process background tasks.
- Startup and shutdown events.
- Test client built on requests.
- CORS, GZip, Static Files, Streaming responses.
- Session and Cookie support.
- 100% test coverage.
- 100% type annotated codebase.
- Few hard dependencies.
- Compatible with asyncio and trio backends.
- Great overall performance against independent benchmarks.



Pydantic

FastAPI Concepts

- Pydantic is a library that allows data validation using Python type hints
"type hints" are a special syntax that allow declaring the type of a variable in Python
- FastAPI uses Pydantic for defining and validating incoming/outgoing data

```
from datetime import datetime
from typing import List, Optional
from pydantic import BaseModel

class User(BaseModel):
    id: int
    name = 'John Doe'
    signup_ts: Optional[datetime] = None
    friends: List[int] = []

external_data = {'id': '123', 'signup_ts': '2017-06-01 12:22', 'friends': [1, '2', b'3']}
user = User(**external_data)
print(user)
#> User id=123 name='John Doe' signup_ts=datetime.datetime(2017, 6, 1, 12, 22) friends=[1, 2, 3]
print(user.id)
#> 123
```

<https://pydantic-docs.helpmanual.io/>



Python Web Framework

FastAPI Concepts

- Django, Flask, and FastAPI are the three most popular web frameworks in Python.
- Django is more sophisticated and is more often used to build large-scale web applications
- Flask and FastAPI are lightweight and suitable for smaller-scale applications

	Github Stars	Github Forks	Launch Year
Flask	55.3k	14.3k	2011
Django	57.5k	24.5k	2006
FastAPI	31.1k	2.2k	2019

Reference:

- FastAPI vs Flask - The Complete Guide ([link](#))



FastAPI vs Flask

FastAPI Concepts

	FastAPI	Flask
HTTP Methods	<code>@app.get("/")</code> <code>@app.post("/")</code>	<code>@app.route("/", methods=["GET"])</code> <code>@app.route("/", methods=["POST"])</code>
Data Validation	Yes	No
Error Message Display	Error displayed in JSON format	Error displayed in HTML pages
Asynchronous	Yes	No
Performance	ASGI	WSGI
Documentation Support	Automatic	Manual

FastAPI Concepts

FastAPI 101

ML Application Example

Agenda.



Terminology

FastAPI 101

Basic Example

```
from fastapi import FastAPI
app = FastAPI()

@app.get('/hello')
def hello():
    return {'message': 'hello'}

@app.get('/ehlo')
def ehlo():
    return {'message': 'ehlo'}
```

Operations

- `app.get()`
- `app.post()`
- `app.put()`
- `app.delete()`

Path

- `/hello`
- `/ehlo`

Path Operation Decorator

- `@app.get('/hello')`
- `@app.get('/ehlo')`

Path Operation Function

- `def hello(): return {...}`
- `def ehlo(): return {...}`



Install FastAPI via pip

FastAPI 101

Pip Installation Commands

- `pip install fastapi`
- `pip install uvicorn` - minimal (pure Python) dependencies
- `pip install uvicorn[standard]` - "Cython-based" dependencies and other "extras"



Create file main.py

FastAPI 101

Minimal Example

Create file `main.py` with content:

```
from fastapi import FastAPI

app = FastAPI()

@app.get('/')
def index():
    return {'message': 'hello world'}
```

Operations

- `app.get()`

Path

- `/`

Path Operation Decorator

- `@app.get('/')`

Path Operation Function

- `def index(): return {...}`



Run app using Uvicorn

FastAPI 101

Using Uvicorn to start server

- `uvicorn` - lightning-fast ASGI server implementation, using `uvloop` and `httptools`
- `main` - name of the module to run (filename without `.py` extension)
- `app` - name of the `FastAPI` class instance (inside of the `main.py` file)
- `--reload` - reload webserver each time when `main.py` changes (flag is optional)

```
$ uvicorn main:app --reload
# INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
# INFO:      Started reloader process [63402] using statreload
# INFO:      Started server process [63404]
# INFO:      Waiting for application startup.
# INFO:      Application startup complete.
```

Optional configuration arguments:

```
$ uvicorn main:app --workers 2
$ uvicorn main:app --reload --port 8000
$ uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Open browser to:

Home

<https://127.0.0.1:8000>

Swagger Doc

<https://127.0.0.1:8000/docs>

ReDoc

<https://127.0.0.1:8000/redoc>

FastAPI Concepts

FastAPI 101

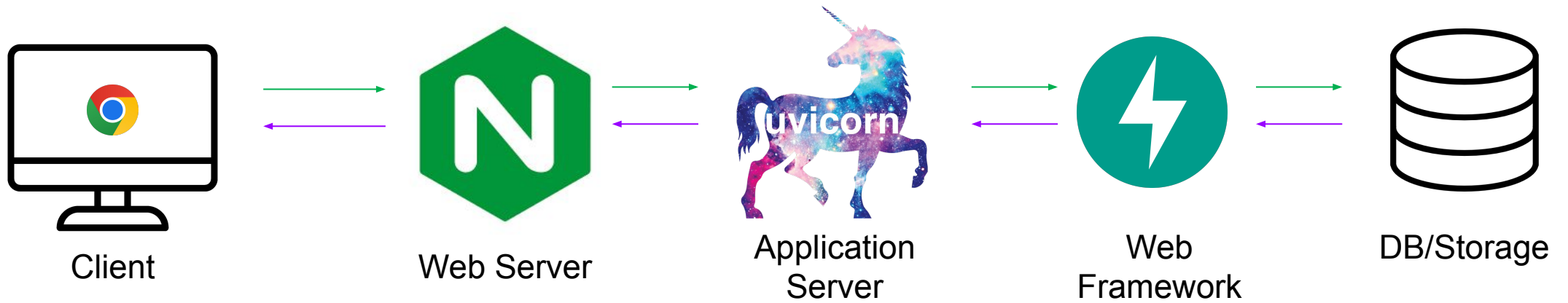
ML Application Example

Agenda.



Web Application Overview

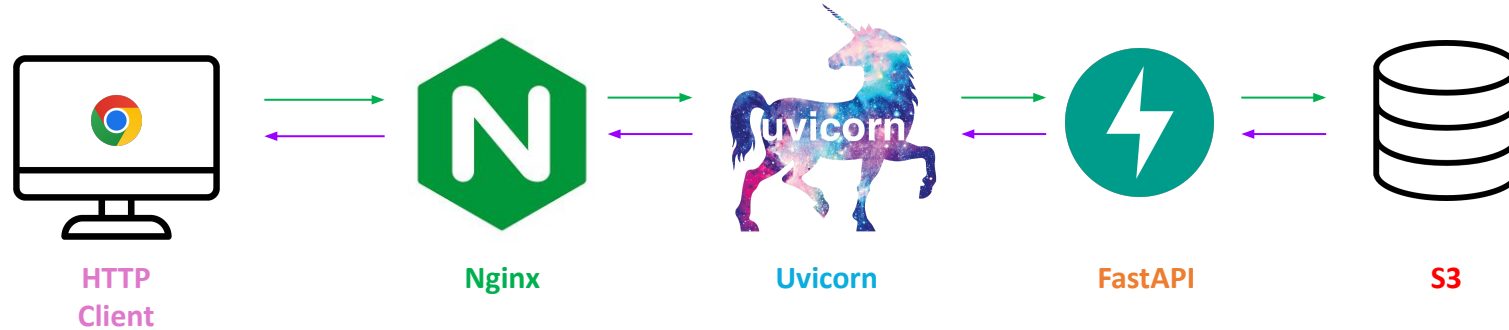
FastAPI Concepts





Web Application Flowchart

FastAPI Concepts



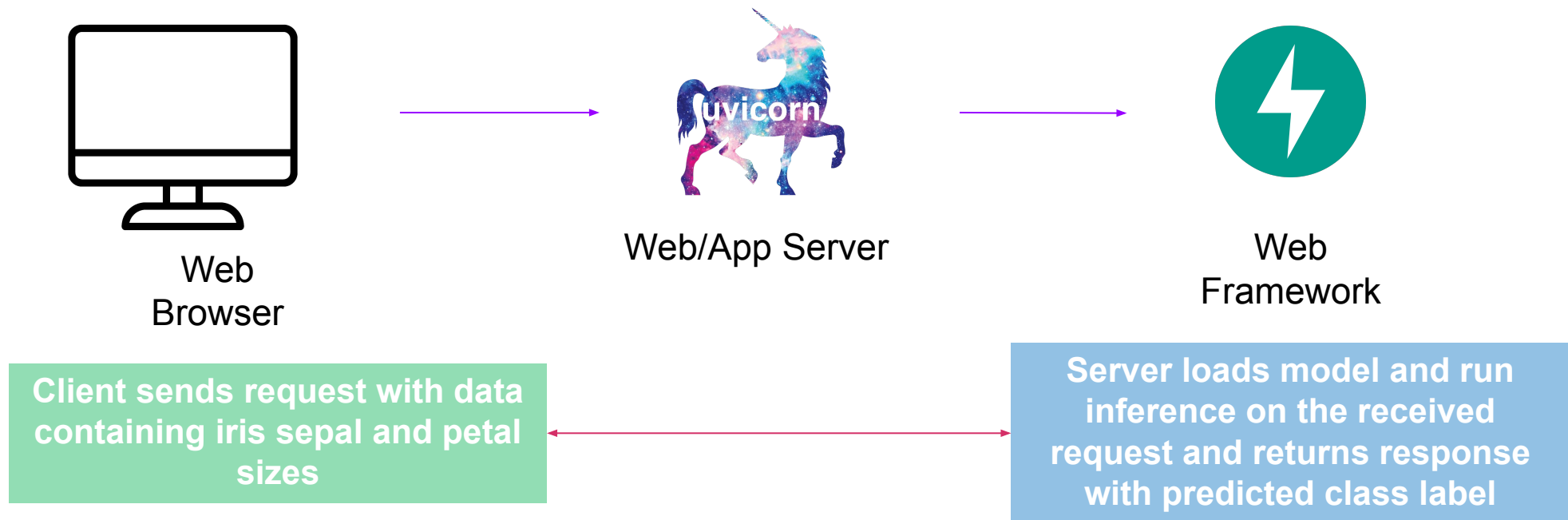
1. A browser (**HTTP Client**) makes an HTTP request to your web app.
2. **Nginx** receives the HTTP request.
 - If it is a static resource(css/js/images), Nginx serves it without bothering **Uvicorn**.
 - If the url is a dynamic one, **Nginx** receives all the TCP packets belonging to the HTTP request efficiently and forwards the request to **Uvicorn** once the entire HTTP request is received. This frees your **Uvicorn** processes from handling slow clients.
3. **Uvicorn** receives the request from **Nginx**, translates it to a **ASGI** compatible request and calls the web framework's request handler method.
4. The **web framework (FastAPI)** implements the business logic and returns an HTTP response.
 - Business logic may include loading model object from **S3** to generate predictions
5. **Uvicorn** now returns the response to **Nginx** which in turn sends the response back to the client.



Iris Classification

ML Application Example

What we are building

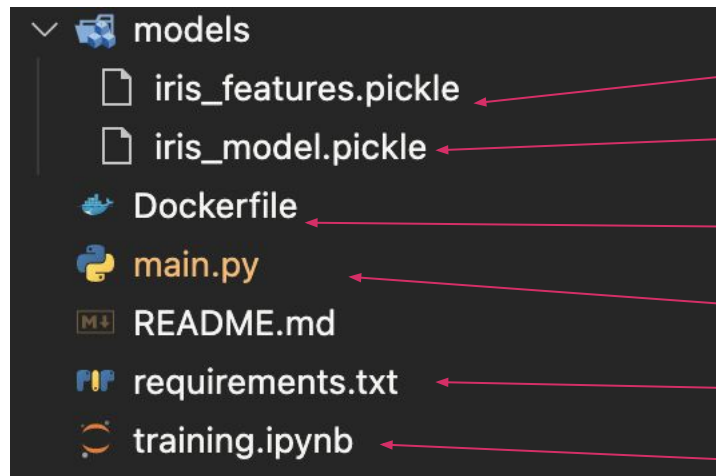




Project Structure

ML Application Example

Project structure



- 1 Model Object
- 2 Feature Names
- 3 Dockerfile
- 4 Application Logic
- 5 Python Dependencies
- 6 Training Notebook to train and save model



Fast API Code

ML Application Example

1

2

3

4

5

6

7

8

```
# Data Handling
import pickle
import numpy as np
from pydantic import BaseModel

# Server
from fastapi import FastAPI

app = FastAPI()

# Initialize files
clf = pickle.load(open('models/iris_model.pickle', 'rb')) # model object
features = pickle.load(open('models/iris_features.pickle', 'rb')) # feature names

class IrisData(BaseModel):
    sepal_length: float
    sepal_width: float
    petal_length: float
    petal_width: float

@app.post("/predict")
def predict(data: IrisData):
    # Extract data into a dictionary
    data_dict = data.dict()

    # Ensure features are in the right order and shape
    to_predict = np.array([data_dict[feature] for feature in features]).reshape(1, -1)

    # Generate prediction
    prediction = clf.predict(to_predict).tolist()

    # Return prediction
    return {"prediction": int(prediction[0])}
```

Application Logic

- 1 Instantiate FastAPI framework
- 2 Load the model object and feature names
- 3 Create data model for incoming data using Pydantic
- 4 Create path route of POST
- 5 Create predict function that contains the application logic
- 6 Predict function reads and preprocess the incoming data
- 7 Model predicts on it
- 8 Application returns prediction



Run Server Locally

ML Application Example

Start server via uvicorn

Command

```
uvicorn main:app --reload --host 0.0.0.0 --port 80
```

Execution

```
$ uvicorn main:app --reload --host 0.0.0.0 --port 80
INFO: Will watch for changes in these directories: ['/Users/schen/wecloud/fastAPI_demo']
INFO: Uvicorn running on http://0.0.0.0:80 (Press CTRL+C to quit)
INFO: Started reloader process [93270] using StatReload
INFO: Started server process [93283]
INFO: Waiting for application startup.
INFO: Application startup complete.
```



Run Server Locally with Docker

ML Application Example

Start server via Docker

- 1 Create a Dockerfile
- 2 Build Docker image
- 3 Run Docker Container

Dockerfile

```
Dockerfile > ...
1  #
2  FROM python:3.9
3
4  #
5  WORKDIR /code
6
7  #
8  COPY ./requirements.txt /code/requirements.txt
9
10 #
11 RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt
12
13 #
14 COPY . /code
15
16 #
17 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "80"]
18
```



Run Server Locally with Docker

ML Application Example

Start server via Docker

- 1 Create a Dockerfile
- 2 Build Docker image
- 3 Run Docker Container

Command

```
# Build a Docker image  
docker build -t wecloud-iris-ml-build-image .
```

Execution

```
$ docker build -t wecloud-iris-ml-build-image .  
[+] Building 2.4s (10/10) FINISHED  
=> [internal] load build definition from Dockerfile  
=> => transferring dockerfile: 37B  
=> [internal] load .dockerignore  
=> => transferring context: 2B  
=> [internal] load metadata for docker.io/library/python:3.9  
=> [1/5] FROM docker.io/library/python:3.9@sha256:8beef516aadb25a759b19eff0ec0bffa47caa19-  
=> [internal] load build context  
=> => transferring context: 1.57kB  
=> CACHED [2/5] WORKDIR /code  
=> CACHED [3/5] COPY ./requirements.txt /code/requirements.txt  
=> CACHED [4/5] RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt
```



Run Server Locally with Docker

ML Application Example

Start server via Docker

- 1 Create a Dockerfile
- 2 Build Docker image
- 3 Run Docker Container

Command

```
# Run image as a Docker container
docker run -d -p 80:80 --name wecloud-iris-api wecloud-iris-ml-build-image
```

Execution

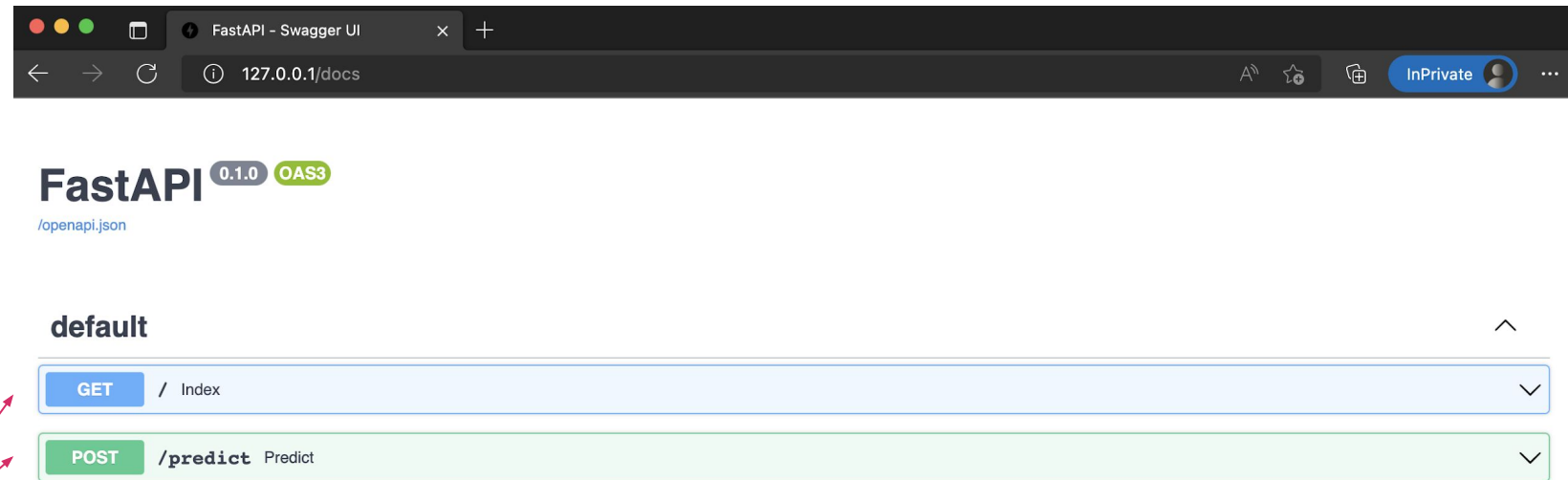
```
(wecloud378)
schen ~/wecloud/fastAPI_demo
$ docker run -d -p 80:80 --name wecloud-iris-api wecloud-iris-ml-build-image
04fc1f3075af8eb5bfe5183d3a6e809ed9f67483f5957ddc6bf20ecad76501f1
(wecloud378)
schen ~/wecloud/fastAPI_demo
$
```



Browser (Client)

ML Application Example

Open Browser at 127.0.0.1/docs



Paths defined
by our
Application
logic



Browser (Client)

ML Application Example

Interacting with /predict path

The screenshot shows a REST client interface for the **POST /predict** endpoint. The interface is divided into several sections:

- Header:** Shows the method **POST** and the path **/predict** with a sub-label **Predict**. A small upward arrow icon is on the right.
- Parameters:** A section titled **Parameters** with a **Try it out** button on the right. Below the title, it says "No parameters".
- Request body:** A section titled **Request body** with a red **required** label. On the right, there is a dropdown menu showing **application/json**.
- Example Value / Schema:** A section with two tabs: **Example Value** and **Schema**. The **Schema** tab is active, displaying a JSON schema in a dark box:

```
{
  "sepal_length": 0,
  "sepal_width": 0,
  "petal_length": 0,
  "petal_width": 0
}
```

Click on this to interact

Request body takes in a JSON

Schema defined by Pydantic in the Application logic



Browser (Client)

ML Application Example

Interacting with /predict path

Model features
and their
respective
values as a
JSON

POST

/predict Predict

^

Parameters

Cancel

Reset

No parameters

Request body required

application/json

▼

```
{
  "sepal_length": 0.7,
  "sepal_width": 0.9,
  "petal_length": 1,
  "petal_width": 2
}
```

Execute

Execute to
send the
request



Request and Response

ML Application Example

Curl Request Structure

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "sepal_length": 0.7,
    "sepal_width": 0.9,
    "petal_length": 1,
    "petal_width": 2
  }'
```

Request URL

```
http://127.0.0.1/predict
```

Response Body

Code

200

Details

Response body

```
{
  "prediction": 0
}
```

Response headers

```
content-length: 16
content-type: application/json
date: Wed,07 Sep 2022 19:36:42 GMT
server: uvicorn
```

FastAPI Demo

Thank You



WeCloudData